

Forkable Sandboxes

The Runtime Layer for AI Software Factories

One factory, seven surfaces, and how to count what a fork tells you

Yossi Eliaz

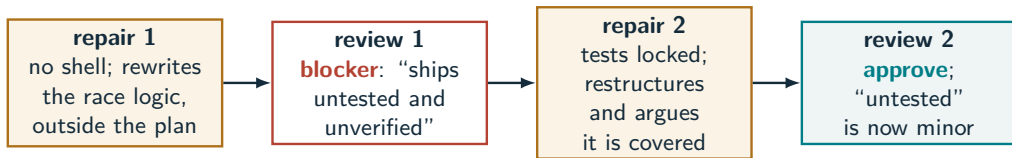
Principal Engineer, Incredibuild · Associate Professor, Holon Institute of Technology (HIT)

Disclosure: I build islo.dev, an agent sandbox platform. I name components; I do not rank vendors.



Blocker. One more repair. Approve.

Work order: fix the sandbox allowlist. A four-thread race test failed; the loop sent a repair agent.



All four steps ran on the same hosted model.

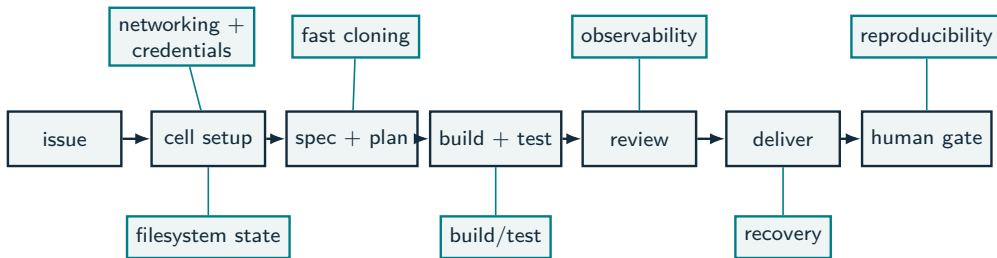
Repair 1: "this environment provided no shell/execution tool, so I could not run" the tests. After it, the harness ran the suite once: 1,972 tests, 0 failures.

27 Sep 2026 · 43 min · \$10.33 · I reverted the file by hand (PR #2351, open)

Every agent told the truth. The gate still approved an untested change.

Run SELFHOST-2: PR #2351 of [zozo123/ariflow-swfactory](#), docs/factory/SELFHOST-2/ (agent/*.json, metrics.json, workgraph-execution.json).

One factory, seven places the runtime bit



MEASURED my run records

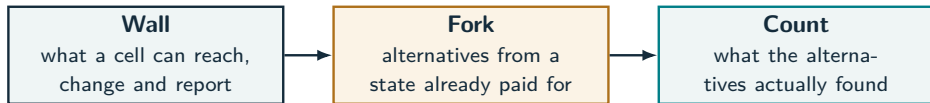
BUILT built and tested, or synthetic

PROPOSED design, not yet run

PUBLISHED other people's work

Scope, said once. Runs SELFHOST-2 and SELFHOST-3 used Docker's sandbox. Two observations that day are from islo: the 403 that became the work order, and a CLI bug that killed an earlier attempt. My harness answered the gates. Leases were off. Nothing forked.

A factory needs a wall, a fork and a count



I claim

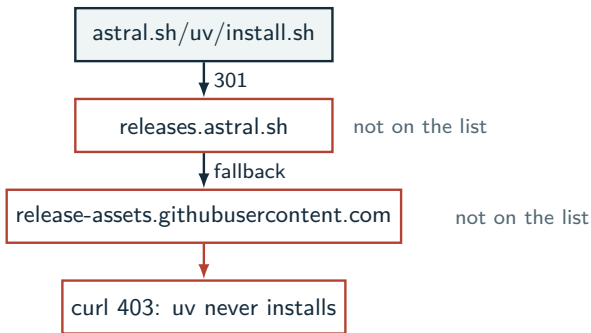
- ✓ a worker/reducer contract: each worker returns estimate, information, n , evidence IDs and lineage, so reused evidence is not counted as agreement; ledger and reducer built (arXiv:2607.09689)
- ✓ real traces from a running factory
- ✓ a hypothesis you can reject: siblings restored from one snapshot fail together more than co-scheduled strangers

I do not claim

- ✗ a new estimator
- ✗ a speed ranking of platforms
- ✗ a factory that forks today
- ✗ a dependence model from lineage

Execution multiplicity is not evidence multiplicity.

Allowlists name hosts; the web redirects



- the list lived in five places: two scripts, a checker, two docs
- the fix added both hosts
- the second one admits every release asset on GitHub

Fetch by digest through a caching proxy, not by name.

Observed on islo, 27 Sep 2026 (islo CLI 0.53.1). Work order: PR #2351, docs/factory/SELFHOST-2/intent.md.

Filesystem state: what a snapshot shares

State	Policy	Known mechanisms
files, build outputs workspace disk	pin by content digest copy-on-write layers	OBuilder: CoW snapshot per build step (ZFS, btrfs) overlay layers (day10); DeltaBox's layered CoW filesystem; chained overlaybd snapshots (DeepSeek DSec)
guest memory	map the image; fault pages on first touch	REAP: prefetch the working set, $3.7\times$ faster cold start
agent history	keep outside the VM	
randomness	reseed, or couple on purpose	slide 22
external services	mediate	slide 15

A snapshot captures a boundary. Measure time to useful work, not time to resume.

OBuilder; day10; DeltaBox; DeepSeek-V4; REAP (ASPLOS'21).

Locking the tests was not enough

In my run

- ✓ tests/ write-protected, and the agent was told
- ✗ planned steps are scope-checked; repairs are not
- ✗ the repair changed the code under test
- ✗ an out-of-plan edit is “major”, not a blocker
- ✗ writer and both reviewers: one model

In published work

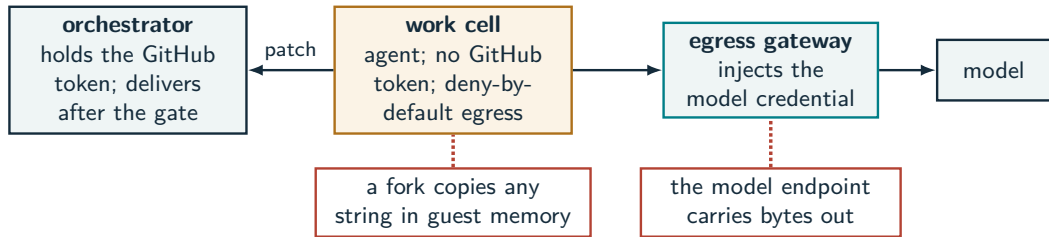
- o3 reward-hacked in 30.4% of RE-Bench runs, 0.7% of HCAST; METR suggests RE-Bench’s visible scorer is why
- read-only tests stop test edits, not special-casing (ImpossibleBench)
- Kimi K3: isolate agents from verifiers; hidden verifiers; limited submission budgets

Two fixes here are policy, not runtime: scope-check repairs; make an out-of-plan edit a blocker.

Run $\neq$ evaluate $\neq$ promote. Each grader gets a budget.

swfactory work_stage.py (repairs recorded without a scope check); METR; ImpossibleBench; Kimi K3 §5.3.2.

The agent never pushes; its token still copies



PROPOSED Next: the gateway keys each request on host-assigned identity (per-clone tap or vsock CID), never on a string in guest memory. Restore bumps a generation and voids the parent's lease.

Copy the machine, not the keys.

swfactory docs/islo.md, docs/credential-authority.md; [Docker Sandboxes security](#); Capsicum (USENIX Sec'10); macaroons (NDSS'14).

Teardown must not destroy the experiment



The stray file was my own harness's review-diff archive, mirrored into the workspace.

Run SELFHOST-3, a second work order: 26 min, \$3.43, nothing delivered.

The gate was right. Recovery was wrong.

The artifact leaves the cell before the cell dies. A human wait should be snapshot, destroy, restore.

SELFHOST-3 run records, 27 Sep 2026; fix in open PR #2350 (harness archives moved to control state).

A green check is not proof the check ran

CI job evals-islo

`success` in 3 s; every real step skipped, because the key was unset

approvals.json

`"mode": "human", actor admin; my harness answered`

islo CLI 0.53.1

status lines on the command's stdout; an earlier attempt died parsing them

All three are my own receipts, and each was wrong.

Every check needs a receipt the child cannot write.

GitHub Actions run 36423283572 (28 Sep 2026); SELFHOST-2 approvals.json; open PR #2348. Provenance: PASS (USENIX ATC'06), CamFlow (SoCC'17), in-toto (USENIX Security'19).

What the run pins, and what it cannot

Pinned in the run record

- ✓ input commit (8ff967d) and `uv.lock`
- ✓ policy and inputs digests
- ✓ sandbox adapter digest
- ✓ every agent step's output, as JSON

Not pinned

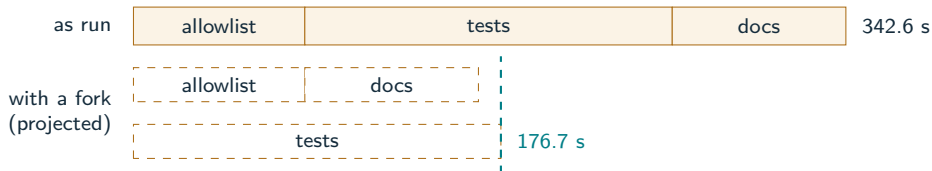
- ✗ the hosted model: no sampling seed
- ✗ external hosts and their redirects
- ✗ the clock and scheduling
- ✗ the adapter's source: a local shim, not in the repo

PROPOSED Replay, not rerun: log every model response, external fetch and clock read; re-execute the rest against the log.

A C2D3 scientist can audit this run. Nobody can rerun it.

SELFHOST-2 metrics.json, workgraph-execution.json. Record and replay: `rr` (USENIX ATC'17); DeepSeek-V4 DSec logs every sandbox command and result for replay ([arXiv:2606.19348](https://arxiv.org/abs/2606.19348)).

Parallel-safe steps, and nothing to fork them



reason: `serial_fallback_missing_fork`

Disjoint files: two worktrees would do here. A fork matters when the reached state is expensive.

Three minutes of forty-three. We fork to buy alternatives.

SELFHOST-2 workgraph-execution.json (node timings measured; the fork lane is computed, fork cost excluded).

Fork is old, and much of it ran on Xen

	System	What it forks	Reported
2003	Xen (Cambridge)	paravirtualised VMs	substrate for most of these
2005	Potemkin	flash-cloned honeypot VMs, CoW memory	521 ms clone to first reply
2009	SnowFlock	VM_fork(N) across hosts, on Xen	600–800 ms; first example forks an untrusted child
2020	Catalyzer	sfork of a running sandbox	under 1 ms, best case
2023	Nephele	fork() for unikernel VMs, on Xen	
2026	DeltaBox; Shepherd	agent sandbox checkpoint; fork	10.83 ms checkpoint; 134–143 ms fork
2026	Kimi K3 AgentENV	Firecracker fork “for reward judging without side effects”	51.2M sandboxes over all K3 runtimes

The mechanism is old. The caller is new: a program that searches.

SOSP'03; SOSP'05; EuroSys'09; ASPLOS'20; EuroSys'23; [DeltaBox](#) (evaluation); [Shepherd](#); [Kimi K3](#) §5.3.2.

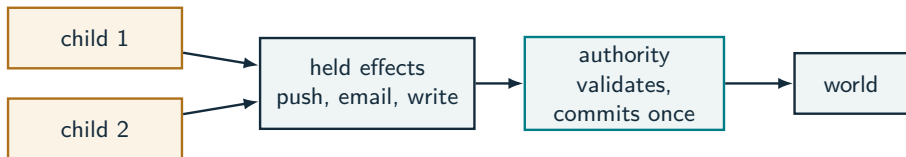
Fork copies memory, secrets and identity

The child inherits	Hazard	What is known
RNG and entropy pool	siblings share random streams	VMGenID reseeds the kernel only; userspace has “no generic solution”
tokens in memory	N live copies	keep secrets outside; wipe on suspend (Brooker et al. 2021)
guest clock	resumes at snapshot time	resync on restore
IP and MAC	identical identities	namespace + NAT per clone; Potemkin spent 142.8 of 521 ms on IP setup
open TCP connections	cannot be cloned	CRIU hands one to one restore; gVisor returns ECONNRESET

Eight forks of a VM holding a token are eight live tokens.

Firecracker snapshotting docs; [Brooker et al., arXiv:2102.12892](#); Vrabl et al. SOSP'05; CRIU and gVisor checkpoint/restore docs.

Hold effects until an authority releases them



- Speculator (SOSP'05), external synchrony (OSDI'06): run ahead, hold output until it is safe
- Remus (NSDI'08): hold network output until the checkpoint commits
- Zheng et al. (2026): an execution edit cannot undo a tool request already sent
- **BUILT** here: children hold no publishing credential; the orchestrator delivers after the gate
- exception: the model API is an unheld channel, metered and logged

Output commit, made structural.

Nightingale, Chen & Flinn SOSP'05; Nightingale et al. OSDI'06; Cully et al. NSDI'08; Zheng et al., [arXiv:2608.22928](https://arxiv.org/abs/2608.22928).

When fork pays, and where it loses

$$(N - 1) P > H + N (R + D)$$

P preparation, H capture, R restore, D divergence,
 N children

Where fork loses

- ✗ $N = 1$
- ✗ the state is only files, and the build cache is good
- ✗ a minimal image boots faster (Jitsu; LightVM)

Restore fan-out, one API round trip

MEASURED

- 141 MB named islo snapshot; restore, run true, capture
- concurrency 12: p50 6.87 s, p95 9.04 s; 255 of 256 succeeded
- per-op teardown excluded; memory capture unverified

Kimi K3: waiting on inference is up to 98% of a sandbox's life. **PUBLISHED**

If your build cache is good, fork loses. Fork pays for state you cannot rebuild.

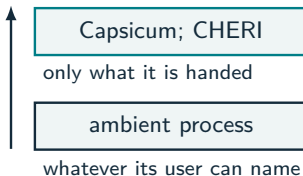
Eliaz, [arXiv:2607.09689](#), Table 2; [Kimi K3](#) §5.3.2; Madhavapeddy et al. NSDI'15; Manco et al. SOSP'17.

Two axes: what a child can reach, and name

Kernel surface



Authority

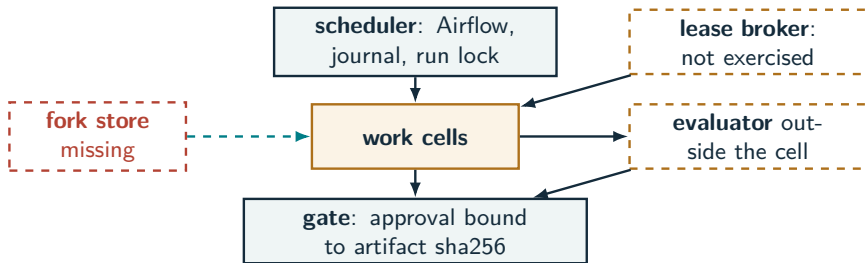


Kimi K3 saw kernel panics in its early container runtimes. Firecracker's production guide: disable SMT and KSM.

Pick the kernel surface for the workload. Hand over authority explicitly.

[Kimi K3 §5.3.2](#); [Firecracker docs/prod-host-setup.md](#); Watson et al., Capsicum (USENIX Sec'10) and CHERI (IEEE S&P'15).

Machines fork; authority stays outside



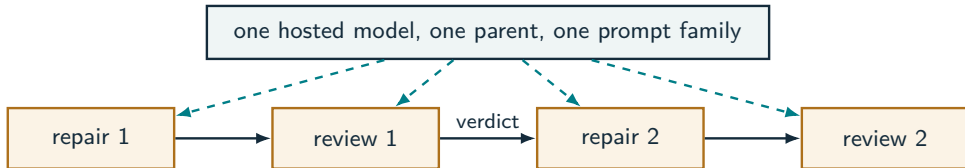
Fork contract, 8 conditions (docs/lifecycle.md), e.g. **5: reused evidence is not independent agreement**; 8: tear down every child.

Promotion is epoch-fenced, like Chubby sequencers and fencing tokens. A TLA+ design model exists, not yet checked against traces.

I wrote the fork contract before I had the fork.

swfactory docs/lifecycle.md, docs/run-recovery.md, formal/authority/; Burrows OSDI'06; Kleppmann (2016).

The second review was not a second witness



- **Sequential:** repair 2 was written against review 1's verdict, then graded by the same model. The candidate adapts to its grader (Dwork et al. 2015).
- **Parallel:** fork the repair loop nine times; nine greens share a parent, a model, a prompt and a flaky test. **PROPOSED**

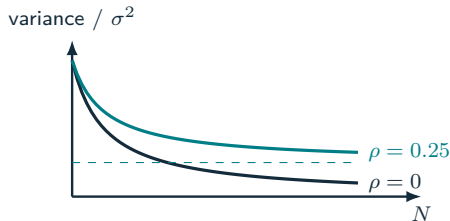
Record both: what feedback each candidate saw, and what siblings share.

Execution multiplicity is not evidence multiplicity

$$\text{Var}(\bar{Y}_N) = \sigma^2 \left(\rho + \frac{1 - \rho}{N} \right)$$

$$N_{\text{eff}} = \frac{N}{1 + (N - 1)\rho}$$

100 forks: 9.2 witnesses at $\rho = 0.1$, 2.0 at $\rho = 0.5$.



On one leaderboard, two models that are both wrong agree 60% of the time (Kim et al., ICML'25; 350+ LLMs studied).

Nine clones are not nine witnesses.

Equal variances, common pairwise correlation $\rho \geq 0$; Kish (1965). Kim et al., [Correlated Errors in Large Language Models, ICML 2025](#).

The runtime holds some of the cluster labels

Methods for dependent evidence

design effect	Kish 1965
cluster-robust variance	Liang & Zeger 1986
dependent effect sizes	Hedges et al. 2010
lineage as annotation	Green et al. PODS'07

All of them need cluster labels.

What lineage can and cannot label

- ✓ shared parent, seed, test, fixture
- ✗ a shared model's blind spots (Kim et al.)
 - parent, model and test are crossed, not nested; a few families are too few clusters for any interval

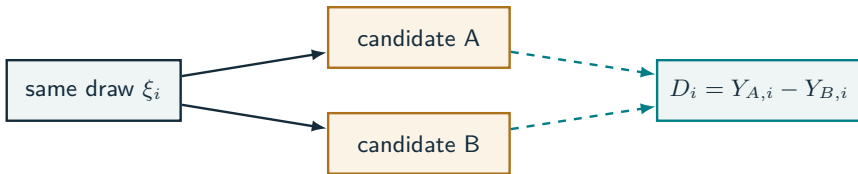
Hand over the labels you have. The dependence model is still open.

Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689), §5 (dependence-aware reduction over fork DAGs remains open).

Fork is also how you run the control

For a comparison, positive covariance helps.

$$\text{Var}(Y_A - Y_B) = \text{Var}(Y_A) + \text{Var}(Y_B) - 2 \text{Cov}(Y_A, Y_B)$$



Share randomness to compare; vary it to corroborate.

The seed policy belongs in the fork API.

Glasserman & Yao, Management Science (1992).

Every worker returns a receipt, not a score

$$r_k = (\hat{\theta}_k, J_k, n_k, \mathcal{E}_k, \mathcal{L}_k, m_k)$$

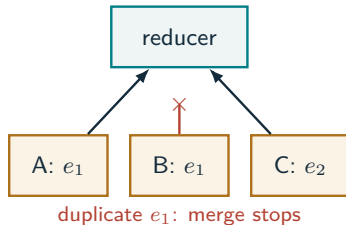
estimate, information, n , evidence IDs, lineage, metadata

BUILT

- numeric summaries merge in any tree order
- a repeated evidence ID stops the merge
- lineage travels with the result; nothing consumes it yet

PROPOSED

- one execution per evidence ID: a retry is never counted twice
- unknown dependence: abstain



duplicate e_1 : merge stops

θ : e.g. one candidate's pass probability; then counts (k, n) add exactly.

MEASURED trace: 4 restores, 6.70 s total; pooled 4.94 vs. 4.95.

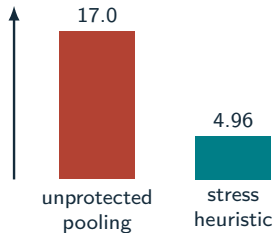
The estimator is Cochran's. The contract is what was missing.

Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689) (2026), §2–5; Cochran (1954).

Isolation does not authenticate precision

One worker reports a distant estimate from a 2,000-point shard and multiplies its precision by a further $50\times$.

pooled estimate



- a sandbox bounds what a child can *do*, not what it can *claim*
- precision ($n \cdot J$) must come from evidence the controller holds: re-run or re-score on held-out data **PROPOSED**
- the heuristic is not a Byzantine guarantee

Precision is an input the child should not control.

Synthetic check, Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689), §4.

The test that could prove me wrong

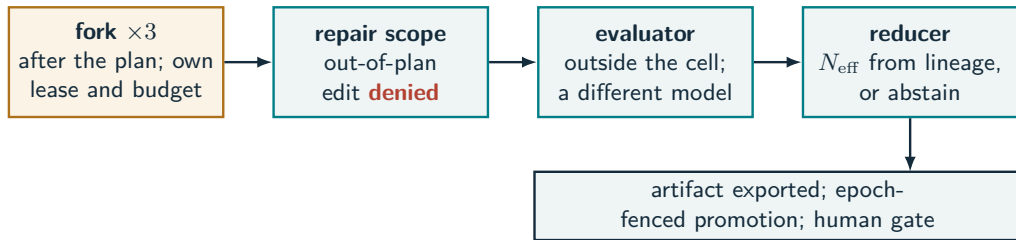
Phase	Design	I predict	Rejects the thesis if
A. cost	cold (no cache), cached template, restore; $N \in \{1, 3, 6, 12\}$; server-side timestamps where exposed	restore beats cold by ≥ 30 s at every N ; if cached is within 10 s at $N=1$, the cache wins	restore does not beat the cached template
B. coupling	6 snapshot families $\times$ 3 siblings vs. 18 co-scheduled cold cells; flaky test $\times 30$ each; no model calls	$\Delta\rho > 0.05$, 95% CI excluding 0	$\Delta\rho \leq 0.05$ and upper CI < 0.1
C. the loop	repair 1 re-sampled 9 times from its recorded input, one model	at least 5 of 9 repeat the out-of-plan edit	descriptive only

Written before any run: PREREGISTRATION.md. Results, held or failed, go in RESULTS.md in the same repository.

A result where the build cache wins is a good result for this room.

github.com/zozo123/cam-talk-london-26.

The same work order, run as it should be



Review 2 would not count as a second witness.

Search can race. Promotion must not.

Open problems where this room is ahead of me

1. **Dependence from lineage.** Estimate ρ from fork lineage and evidence manifests.
2. **Leases that fork.** Capabilities re-minted on restore, revocable, never ambient.
3. **Warm state and side channels.** Pack forks by trust, not only by utilisation.
4. **Evidence commit.** Held effects for remote services and for people.
5. **Layers or VM fork.** When does a content-addressed build beat a live fork?
6. **Human-gate latency.** Snapshot, destroy and restore across a day-long wait.

I would like help with the first two.

Fork the machine, not the trust.

State without secrets.

Graders the children cannot write.

Receipts that outlive the cell.

Evidence counted, not executions.

Authority committed once.

Burn the cell. Keep the receipt.

Help wanted: estimating ρ from lineage; leases that fork.

arXiv:2607.09689 · github.com/zozo123/ariflow-swfactory

github.com/zozo123/cam-talk-london-26 (slides, notes, pre-registration)

Paper Table 2: exercised API paths, not a provider ranking

Platform	State source	Measured operation	Conc.	Batch	Success	p50	p95
Daytona	default environment	create	8	1024	1024/1024	0.20 s	1.12 s
Tensorlake	default environment	create	8	256	256/256	3.44 s	9.00 s
islo	named 141 MB snapshot	restore + run + capture	12	256	255/256	6.87 s	9.04 s

Total wall time: 190.3 s, 387.3 s, 240.3 s. Operation boundaries differ; rows are not comparable as latencies. The islo percentiles exclude the per-op teardown, which shares the slot: 256 ops over 12 slots averaged 11.3 s per slot including teardown. Whether the snapshot captures memory is unverified, so this is restore fan-out.

Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689), Table 2.

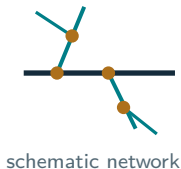
Agent checkpoint and fork, 2026

System	What it forks or restores	Reported figure
DeltaBox	layered CoW filesystem + incremental CRIU in Firecracker	evaluation: 10.83 ms checkpoint, 1.86 ms restore (abstract: 14 ms / 5 ms)
Shepherd	overlay filesystem + backend-native checkpoints	fork 134–143 ms, flat from 42 MB to 5.8 GB
Crab	eBPF effect classification per agent turn	over 75% of turns need no checkpoint
Planarian	statepoints spanning local and remote state	remote rollback by compensating actions
Zheng et al.	when checkpoint, fork, restore, merge are safe	a sent request cannot be undone (Lean)
Kimi K3 AgentENV	Firecracker pause, fork, snapshot for agentic RL	checkpoint as low as 133 ms, resume as low as 49 ms

[2605.22781](#); [2605.10913](#); [2604.28138](#); [2609.35366](#); [2608.22928](#); [Kimi K3 §5.3.2](#).

Arp2/3 shapes network dynamics

In actomyosin simulations, the effect of Arp2/3 depends on concentration.



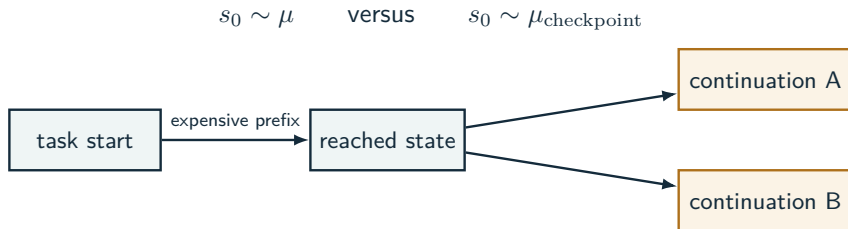
Arp2/3 regime	Reported network response
High concentration	Network stalls
Low concentration	Network contracts
Intermediate	Loosely connected clusters can collapse in motor-driven avalanches

Connectivity and interaction matter alongside branch count.

Li et al. (JPCB, 2021); Eliaz et al. (PRE, 2020); Stockmayer (JCP, 1943).

Reset changes the learning problem

The distribution of starting states determines which experience is collected.

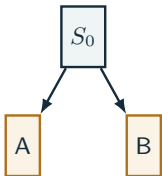


- Record the origin and selection rule for each checkpoint.
- Evaluate the final policy on the intended task distribution.

Related return-and-explore strategy: [Ecoffet et al., Go-Explore, Nature \(2021\)](#).

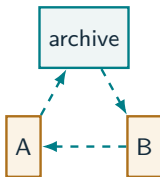
Three graphs describe the computation

Execution ancestry



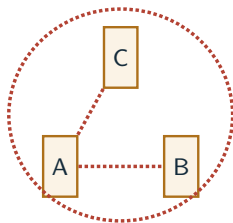
Which state produced this continuation?

Evidence flow



Which observations influenced a decision?

Composition



Which candidate sets need joint validation?

A runtime tree can support a communication network and higher-order interactions.

What does a message contribute?

A branch report matters relative to the decision and evidence already available.

$$I(H; E_k \mid E_{<k})$$

A small synergy example

Let A, B be independent fair bits and $H = A \oplus B$.

$$I(H; A) = I(H; B) = 0, \quad I(H; A, B) = 1 \text{ bit.}$$

For a branch controller

- Redundant messages can add little.
- Complementary messages can matter jointly.
- Estimation requires a distribution and target.

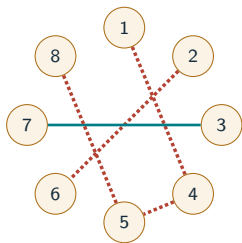
Information guides search; objective, cost and validity decide.

Information decomposition perspective: Williams & Beer, Nonnegative Decomposition of Multivariate Information (2010).

How eight changes work together

Individual success does not establish the quality of a composition.

$$\Gamma_{ij} = U(\{i, j\}) - U(\{i\}) - U(\{j\}) + U(\emptyset)$$



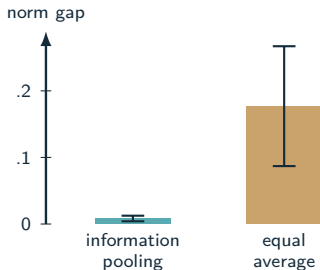
Illustrative interaction graph

- $U(A)$: fixed objective for change set A from the same parent.
- Eight candidates yield 28 pair tests.
- Higher-order effects can survive pairwise screening.
- Rebuild and evaluate each chosen composition.

What the reducer experiments establish

Unequal logistic shards

Distance to the centralized MLE; lower is closer.



0.0083 ± 0.0042 vs. 0.177 ± 0.090

Mean $\pm$ SD over eight fixed seeds.

Sample-size weighting not yet compared (paper §4). Reported synthetic results: [Eliaz \(2026\)](#), [arXiv:2607.09689](#). Error bars show SD, not confidence intervals.

Scope of the checks

- Gaussian summaries match the stated pooling algebra.
- Information weighting helps in this synthetic comparison.
- Forged precision exposes a trust failure; a heuristic is not a robustness theorem.

Ablations that explain the outcome

Vary	Observe	Question
Prefix cost and dirty pages	Resource use and memory	When does reuse pay?
No sharing / archive / routing	Quality at equal full cost	When does communication help?
Evidence overlap and coupling	Calibration and error	Does the reducer overcount?
Evaluation budget	False acceptance and time to target	Where should compute go?
Composition order and size	Joint success and regressions	Which interactions matter?

Separate a runtime gain from a search-policy or evaluation advantage.

Evaluation under adaptive search

A protected evaluator can still teach the search how to overfit its feedback.



- Keep evaluation code and admission records outside worker write access.
- Track queries and selection history.
- Use fresh validation or a justified adaptive evaluation procedure.

Russo & Zou, Controlling Bias in Adaptive Data Analysis Using Information Theory, AISTATS (2016).

The fork contract, before the fork

A future native-fork executor must satisfy all of these before enabling parallel branches:

1. fork from an immutable, content-addressed parent state
2. give every branch its own lease, TTL, budget and provenance
3. never copy publishing credentials into a coding cell
4. record the parent checkpoint, branch ID and evidence IDs for every child
5. **never count siblings that reuse the same evidence as independent agreement**
6. merge deterministically and surface conflicts as evidence, not silent retries
7. run the complete verification and review stream on the merged result
8. tear down every child on success, failure, cancellation or scheduler recovery

swfactory [docs/lifecycle.md](#).

References: systems

Speaker

Y. Eliaz. Evidence-aware reduction for forkable compute. arXiv:2607.09689, 2026.
github.com/zozo123/ariflow-swfactory, runs SELFHOST-2/3, 27 Sep 2026.

Fork, snapshot, isolation

Barham et al. Xen and the art of virtualization. SOSP 2003.
Vrable et al. Scalability, fidelity, and containment in the Potemkin virtual honeyfarm. SOSP 2005.
Lagar-Cavilla et al. SnowFlock. EuroSys 2009.
Madhavapeddy et al. Jitsu. NSDI 2015.
Manco et al. My VM is lighter (and safer) than your container. SOSP 2017.
Baumann et al. A fork() in the road. HotOS 2019.
Agache et al. Firecracker. NSDI 2020.
Du et al. Catalyzer. ASPLOS 2020.
Brooker et al. Restoring uniqueness in microVM snapshots. arXiv:2102.12892, 2021.
Ustiugov et al. Benchmarking, analysis, and optimization of serverless function snapshots (REAP). ASPLOS 2021.
Nephele: extending virtualization environments for cloning unikernel-based VMs. EuroSys 2023.

Agent sandboxes, 2026

DeltaBox, arXiv:2605.22781. Shepherd, arXiv:2605.10913. Crab, arXiv:2604.28138.
Planarian, arXiv:2609.35366. Zheng et al., arXiv:2608.22928.
Kimi K3 technical report (AgentENV), 2026. DeepSeek-V4 (DSec), arXiv:2606.19348.

Effects, authority, provenance

Gray & Cheriton. Leases. SOSP 1989.
Nightingale, Chen & Flinn. Speculative execution in a distributed file system. SOSP 2005.
Nightingale et al. Rethink the sync. OSDI 2006.
Cully et al. Remus. NSDI 2008.
Watson et al. Capsicum. USENIX Security 2010.
Watson et al. CHERI. IEEE S&P 2015.
Birgisson et al. Macaroons. NDSS 2014.
Burrows. The Chubby lock service. OSDI 2006.
Muniswamy-Reddy et al. Provenance-aware storage systems. USENIX ATC 2006.
Pasquier et al. Practical whole-system provenance capture. SoCC 2017.
Torres-Arias et al. in-toto. USENIX Security 2019.
O'Callahan et al. Engineering record and replay for deployability. USENIX ATC 2017.

References: evidence and evaluation

Dependent evidence

Cochran. The combination of estimates from different experiments. *Biometrics* 1954.

Kish. *Survey Sampling*. Wiley, 1965.

Liang & Zeger. Longitudinal data analysis using generalized linear models. *Biometrika* 1986.

Hedges, Tipton & Johnson. Robust variance estimation in meta-regression with dependent effect size estimates. *Research Synthesis Methods* 2010.

Green, Karvounarakis & Tannen. Provenance semirings. *PODS* 2007.

Glasserman & Yao. Some guidelines and guarantees for common random numbers. *Management Science* 1992.

Kim et al. Correlated errors in large language models. *ICML* 2025.

Evaluation under search

METR. Recent frontier models are reward hacking. June 2025.

Zhong, Raghunathan & Carlini. ImpossibleBench. *arXiv:2510.20270*, 2025.

Dwork et al. The reusable holdout. *Science* 2015.

Stroebl, Kapoor & Narayanan. Inference scaling fLaws. *arXiv:2411.17501*, 2024.

Russo & Zou. Controlling bias in adaptive data analysis using information theory. *AISTATS* 2016.

Backup material

Liman et al. *PNAS* 2020; Eliaz et al. *PRE* 2020; Li et al. *JPCB* 2021 (Arp2/3).

Ecoffet et al. Go-Explore. *Nature* 2021.

Williams & Beer. Nonnegative decomposition of multivariate information. 2010.